



ELL201 - Project

Designing Counters

ATHARVA DESHMUKH

2024EE30692

YASH GARG

2024EE10175

AM GORE

2024EE10299

ARNAB MANDAL

2024EE10454

Date started : 7 Nov 2025

Date Finished & Verified : 14 Nov 2025

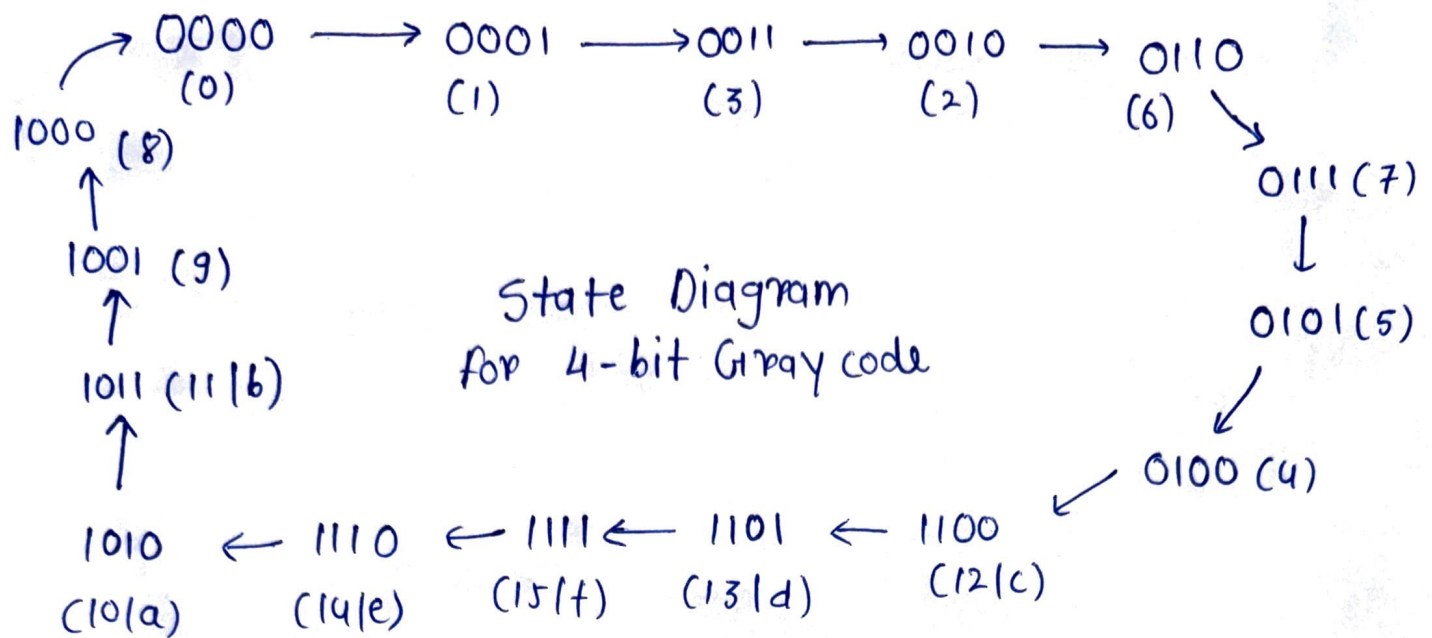
Date of Report Submission : 15 Nov 2025

TA: RRSSD Prasad.

Brief

Apparatus/Materials Used:

- We used Verilog to code our counter.
- we used Xilinx Artix-7 FPGA Board for application.
- USB A-C for bitstream transfer.



Since States are till 15, we need 4 bits to represent it and hence 4 Flip Flops are needed.

STATE TABLE for Gray Counter (4-bit)

(Current State)				(Next state)											
Q_3	Q_2	Q_1	Q_0	D_3	D_2	D_1	D_0	S_3	R_3	S_2	R_2	S_1	R_1	S_0	R_0
0	0	0	0	0	0	0	1	0	X	0	X	0	X	1	0
0	0	0	1	0	0	1	1	0	X	0	X	1	0	X	0
0	0	1	0	0	1	1	0	0	X	1	0	X	0	0	X
0	0	1	1	0	0	1	0	0	X	0	X	X	0	0	1
0	1	0	0	1	1	0	0	1	0	X	0	0	X	0	X
0	1	0	1	0	1	0	0	0	X	X	0	0	X	0	1
0	1	1	0	0	1	1	1	0	X	X	0	X	0	1	0
0	1	1	1	0	1	0	1	0	X	X	0	0	1	X	0
1	0	0	0	0	0	0	0	0	1	0	X	0	X	0	X
1	0	0	1	1	0	0	0	X	0	0	X	0	X	0	1
1	0	1	0	1	0	1	1	X	0	0	X	X	0	1	0
1	0	1	1	1	0	0	1	X	0	0	X	0	1	X	0
1	1	0	0	1	1	0	1	X	0	X	0	0	X	1	0
1	1	0	1	1	1	1	1	X	0	X	0	1	0	X	0
1	1	1	0	1	0	1	0	X	0	0	1	X	0	0	X
1	1	1	1	1	1	1	0	X	0	X	0	X	0	0	1

For any SR flip-flop:

Q (current state)	D (next state)	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

State equation: $Q_{n+1} = S + \bar{R}Q_n$

Reducing the expressions by k-maps:

• S_3 :

$Q_3 Q_2$ \ $Q_1 Q_0$	00	01	11	10
00				
01	1			
11	x	x	x	x
10		x	x	x

$$S_3 = Q_2 \cdot \bar{Q}_1 \bar{Q}_0$$

R_3 :

$Q_3 Q_2$ \ $Q_1 Q_0$	00	01	11	10
00	x	x	x	x
01		x	x	x
11				
10	1			

$$R_3 = \bar{Q}_2 \cdot \bar{Q}_1 \bar{Q}_0$$

• S_2 :

$Q_3 Q_2$ \ $Q_1 Q_0$	00	01	11	10
00				1
01	x	x	x	x
11	x	x	x	
10				

$$S_2 = \bar{Q}_3 Q_1 \bar{Q}_0$$

R_2 :

$Q_3 Q_2$ \ $Q_1 Q_0$	00	01	11	10
00	x	x	x	
01				
11				1
10	x	x	x	x

$$R_2 = Q_3 Q_1 \bar{Q}_0$$

S_1 :

$Q_3 Q_2$ \ $Q_1 Q_0$	00	01	11	10
00	1	x	x	
01				x
11	1	x	x	x
10				x

$$S_1 = \bar{Q}_3 \bar{Q}_2 Q_0 + Q_3 Q_2 Q_0$$

R_1 :

$Q_3 Q_2$ \ $Q_1 Q_0$	00	01	11	10
00	x			
01	x	x	1	
11	x			
10	x	x	1	

$$R_1 = \bar{Q}_3 Q_2 Q_0 + Q_3 \bar{Q}_2 Q_0$$

S_0 :

$Q_3 Q_2$ \ $Q_1 Q_0$	00	01	11	10
00	1	x		
01			x	1
11	1	x		
10			x	1

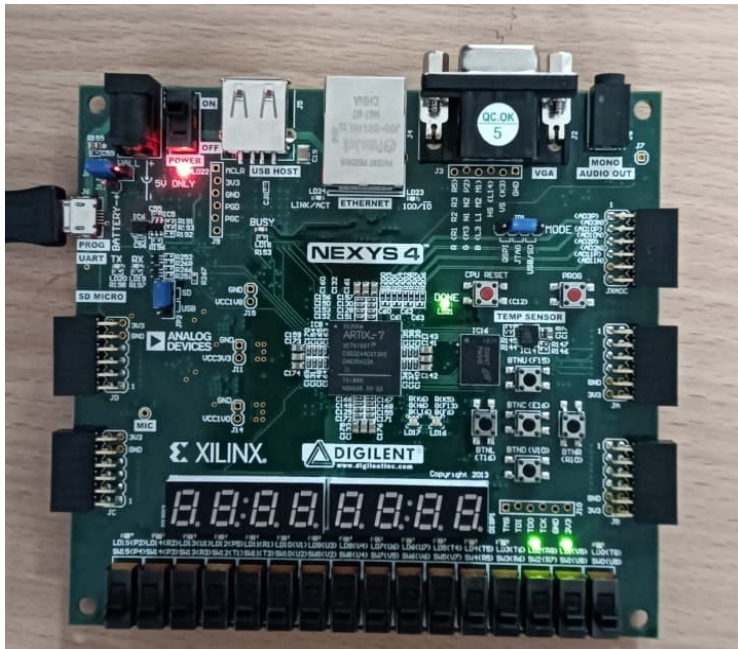
$$S_0 = (Q_3 Q_2 + \bar{Q}_3 \bar{Q}_2) \cdot \bar{Q}_1 + (Q_3 \bar{Q}_2 + \bar{Q}_3 Q_2) \cdot Q_1$$

R_0 :

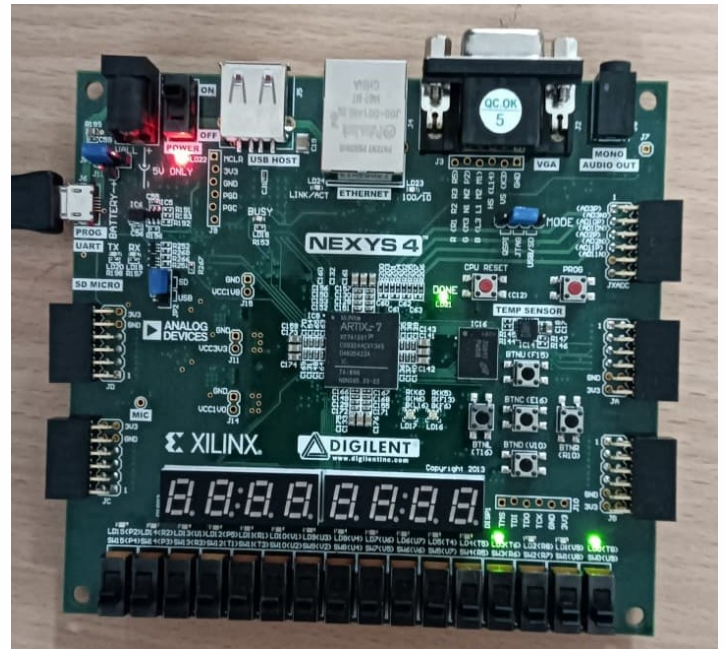
$Q_3 Q_2$ \ $Q_1 Q_0$	00	01	11	10
00			1	x
01	x	1		
11			1	x
10	x	1		

$$R_0 = (Q_3 Q_2 + \bar{Q}_3 \bar{Q}_2) \cdot Q_1 + (Q_3 \bar{Q}_2 + \bar{Q}_3 Q_2) \cdot \bar{Q}_1$$

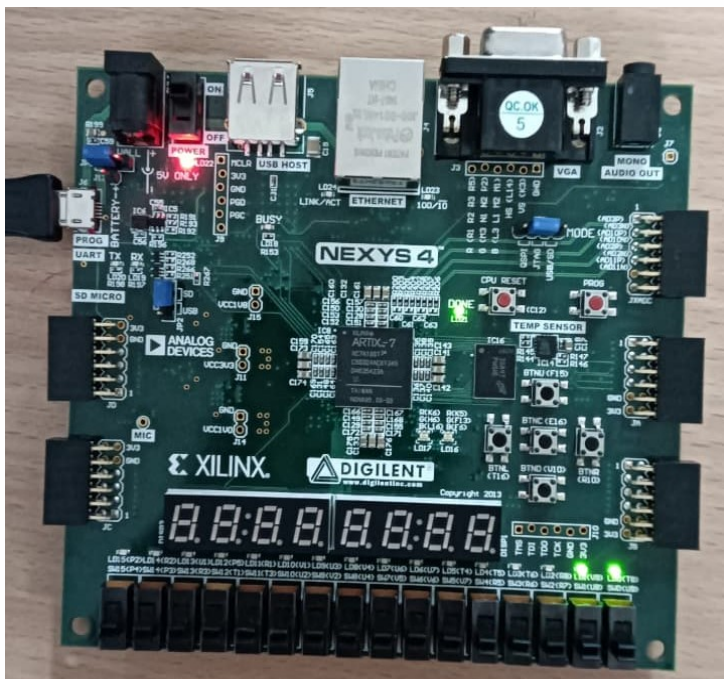
FPGA Board Images (Refer Video for all 16 outputs)



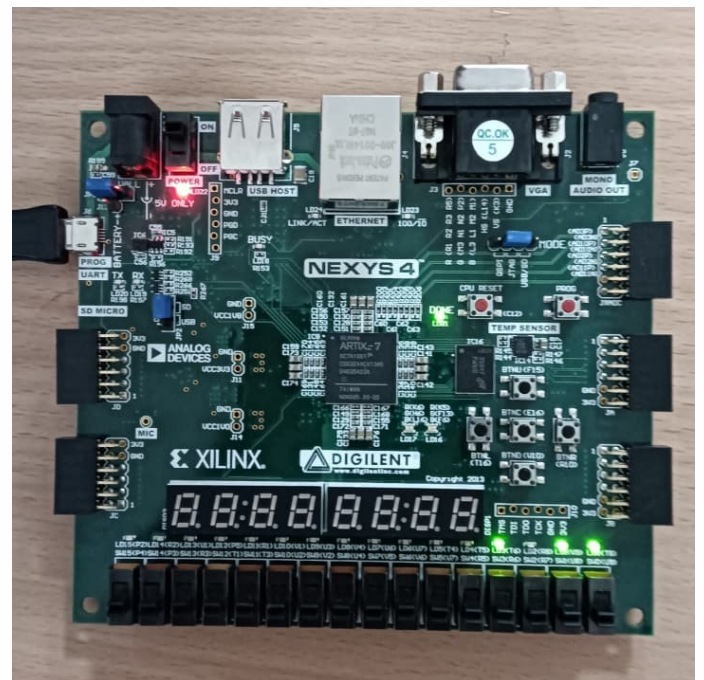
0110



1001

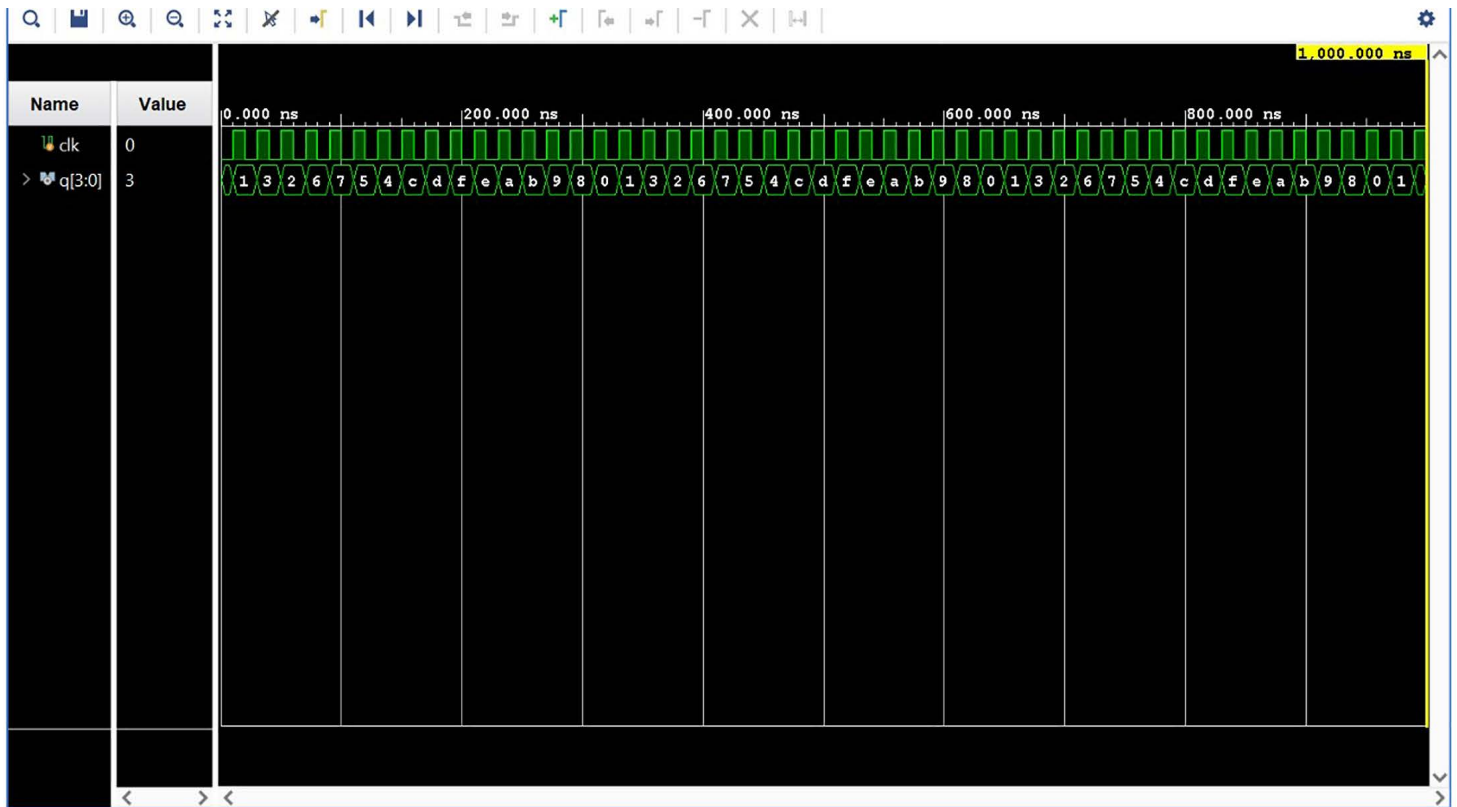


0011

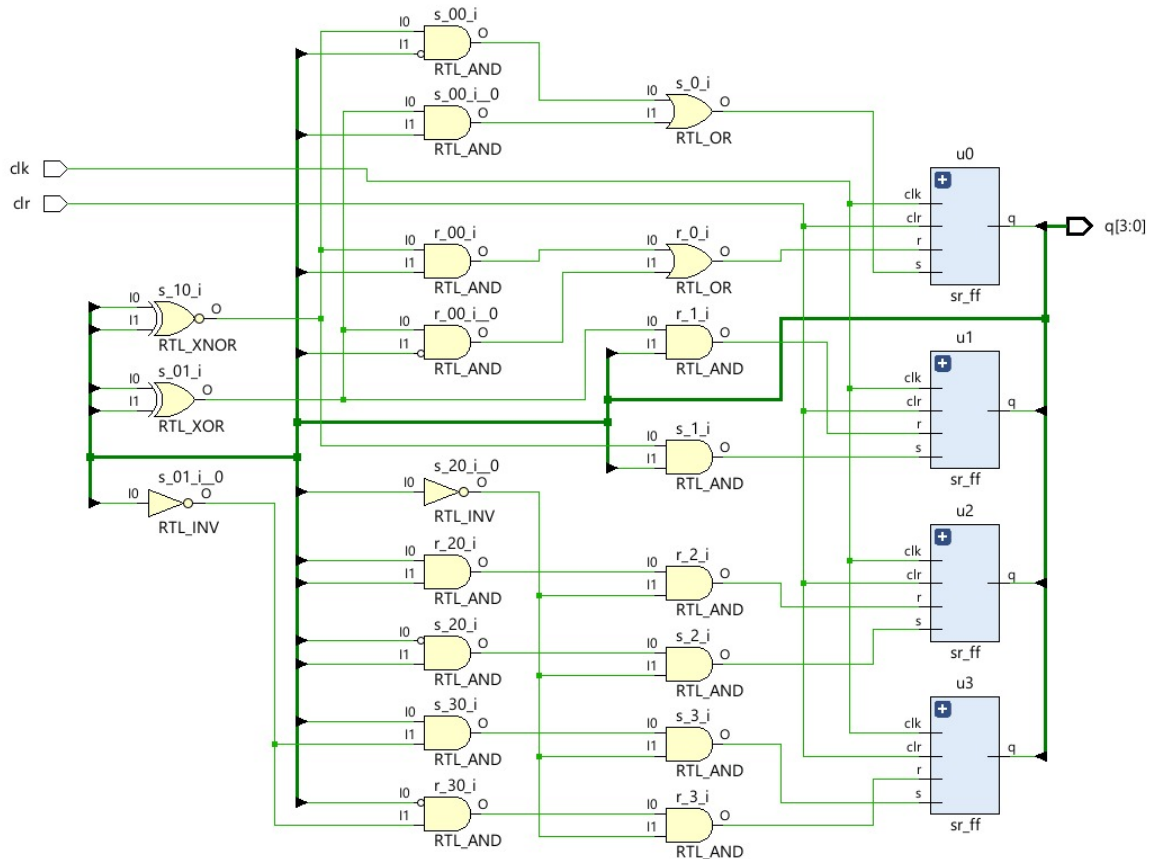


1011

Behavioral Simulation (Waveforms)



Circuit Representation



Verilog Code for 4 bit Gray counter

Table 15 Friday Batch

November, 2025

Implementation Video in Nexys-4 Board

Design sr_ff.v

Listing 1: Verilog design for a SR flip flop

```
1 `timescale 1ns / 1ps
2
3 module sr_ff(
4     input s,
5     input r,
6     input clk,
7     output reg q,
8     input clr //asynchronous //active high
9 );
10
11     initial begin
12         q <= 0;
13     end
14
15     always@(posedge clk or posedge clr) begin
16         if( clr) q <= 0; //asynchronous clear
17         else begin
18             case({s,r})
19                 2'b00 : q <= q; //hold
20                 2'b01 : q <= 0; //set
21                 2'b10 : q <= 1; //reset
22                 2'b11 : q <= 1'bx; //undefined behaviour // race condition
23             endcase
24         end
25     end
26 endmodule
```

Design (gray_counter.v)

Listing 2: Verilog module for 4 bit Gray counter design

```
1 `timescale 1ns / 1ps
2
3 module gray_counter(
4     input clk,
5     input clr,
6     output [3:0] q
7 );
8
9     wire [3:0] s;
10    wire [3:0] r;
11
12    //uncomment all this for board implementation
13    reg [30 : 0] count = 0;
14    reg slow_clk; //100 MHz -> 1Hz
15
16    always @(posedge clk) begin
17        if(count > 99_999_999 — clr == 1) begin
18            count <= 0;
19            slow_clk <= ~slow_clk;
```

```

20     end
21
22     else begin
23         count <= count+ 1;
24     end
25 end
26 //
27
28
29 assign s[3] = q[2] & ~q[1] & ~q[0];
30 assign r[3] = ~q[2] & ~q[1] & ~q[0];
31 assign s[2] = ~q[3] & q[1] & ~q[0] ;
32 assign r[2] = q[3] & q[1] & ~q[0];
33 assign s[1] = (q[3] ^ q[2]) & q[0];
34 assign r[1] = (q[3] ^ q[2]) & q[0];
35 assign s[0] = ( (q[3] ^ q[2]) & ~q[1] ) — ((q[3] ^ q[2]) & q[1]);
36 assign r[0] = ( (q[3] ^ q[2]) & q[1] ) — ((q[3] ^ q[2]) & ~q[1]);
37
38
39 sr_ff u0(.s(s[0]) , .r(r[0]) , .clk(slow_clk) , .q(q[0]) , .clr(clr));
40 sr_ff u1(.s(s[1]) , .r(r[1]) , .clk(slow_clk) , .q(q[1]) , .clr(clr));
41 sr_ff u2(.s(s[2]) , .r(r[2]) , .clk(slow_clk) , .q(q[2]) , .clr(clr));
42 sr_ff u3(.s(s[3]) , .r(r[3]) , .clk(slow_clk) , .q(q[3]) , .clr(clr));
43
44 endmodule

```

Testbench (gray_counter_tb.v)

Listing 3: Testbench for the 4 bit Gray counter

```

1 `timescale 1ns / 1ps
2
3 module gray_counter_tb ;
4
5     reg clk;
6     reg clr = 0;
7     wire [3:0] q;
8
9     initial begin
10         clk = 0;
11         forever #10 clk <= ~clk;
12     end
13
14     gray_counter uut(.clk(clk) , .clr(btnC) , .q(q));
15
16 endmodule

```

Top module (top.v)

Listing 4: Top module for implementation

```

1 `timescale 1ns / 1ps
2
3 module top(
4     input clk ,
5     input btnC,
6     output [3:0] q
7 );
8
9     gray_counter uut (.clk(clk) , .clr(btnC) , .q(q));
10 endmodule

```

Constraints (nexys4_gray_counter.xdx)

Listing 5: Constraints for the 4 bit Gray counter implementation

```

1 ## Clock signal

```



```

2 ##Bank = 35, Pin name = IO_L12P_T1_MRCC_35,          Sch name = CLK100MHZ
3 set_property PACKAGE_PIN E3 [get_ports clk]
4   set_property IOSTANDARD LVCMOS33 [get_ports clk]
5   create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]
6
7 ## LEDs
8 ##Bank = 34, Pin name = IO_L24N_T3_34,                Sch name = LED0
9 set_property PACKAGE_PIN T8 [get_ports {q[0]}]
10   set_property IOSTANDARD LVCMOS33 [get_ports {q[0]}]
11 ##Bank = 34, Pin name = IO_L21N_T3_DQS_34,           Sch name = LED1
12 set_property PACKAGE_PIN V9 [get_ports {q[1]}]
13   set_property IOSTANDARD LVCMOS33 [get_ports {q[1]}]
14 ##Bank = 34, Pin name = IO_L24P_T3_34,                Sch name = LED2
15 set_property PACKAGE_PIN R8 [get_ports {q[2]}]
16   set_property IOSTANDARD LVCMOS33 [get_ports {q[2]}]
17 ##Bank = 34, Pin name = IO_L23N_T3_34,                Sch name = LED3
18 set_property PACKAGE_PIN T6 [get_ports {q[3]}]
19   set_property IOSTANDARD LVCMOS33 [get_ports {q[3]}]
20
21 ##Buttons
22 ##Bank = 15, Pin name = IO_L11N_T1_SRCC_15,           Sch name = BTNC
23 set_property PACKAGE_PIN E16 [get_ports btnC]
24   set_property IOSTANDARD LVCMOS33 [get_ports btnC]

```



Details

In our project, we designed and implemented a 4-bit Gray code counter. It cycles through all the states (16) one by one, while ensuring each consecutive state differs by exactly one bit.

The steps include constructing a clean state table for all 16 states in cyclic order.

Since there were 4 bits needed to implement the 16 states, we used 4 SR Flip Flops. Using Karnaugh maps, we minimized the logic expressions for each S and R input.

Finally, a structured verilog implementation using S-R Flip Flops was created. We made a synchronous counter.

Inferences

The task emphasizes understanding of synchronous designs and the advantages of controlled state transitions.

Since Gray code changes only one bit at a time, error occurrences are minimized since there are ~~less~~ no chances of intermediate value while switching or time delay errors possible.

As we used S-R Flip flops even though they are prone to race condition because we never enter the race condition state ($S=1, R=1$) in our implementation.

K-map minimization helped us convert complex logic



Date ____ / ____ / ____

into optimized logic.

We also learnt how to design good in HDL, (verilog).

Summary

We successfully implemented the Synchronous Gray-code counter using SR flip flops and minimized logic using K-maps. We also implemented the same on our CPLD board with verilog.

Precautions

- Ensure each transition differs by one-bit.
- Use SR flip flops and not latches.
- Avoid $S=1, R=1$ conditions.

Sources of Error

- Incorrect Gray sequence.
- Race condition ($S=1, R=1$) due to incorrect logic.
- Errors in K-map minimizations.